

## SPOT CHECK

SKATESPOTS FIELD MANUAL · Nº04

# THE CONFIGURABLE & COMPONENT

How `Button` and `Badge` replaced a growing pile of near-identical one-off components with a small number of reusable ones, driven by typed props.

6 chapters

Glossary

Self-test

Phase 4 — Remaining UI

## ► CONTENTS (6 CHAPTERS + APPENDICES)

# 01 The core problem: component sprawl

### ● PATTERN

KEYWORDS

component sprawl

duplication

### — WHAT

`AddNewSpotButton` existed as its own component, and Save / Cancel / Directions / Edit / Reviews were all on track to become more of the same — a new file per button, each a near-copy of the last.

### — HOW

List every planned button/badge and write two columns: what's *identical* across all of them (box shape, radius, font, cursor) and what's actually *different* (label, fill color,

proportion). The identical column becomes the shared component; the different column becomes props.

#### — WHY

Every one-off component like this duplicates markup and CSS that's ~95% identical for a difference that's actually tiny. That's the same "hand-typed value repeated everywhere" problem as un-tokenized colors — just one level up, at the *component* scale instead of the CSS-value scale.

#### ◆ IDEA TO CONSIDER

Sprawl is easiest to catch early, before the third or fourth near-duplicate exists. The moment you're about to create `CancelButton` after already having `SaveButton`, that's the signal — not after you've built six of them.

## 02 Identity props vs. layout props

### ● PATTERN

KEYWORDS `variant` `inline style` `component API design`

#### — WHAT

`Button` ended up with two props that look similar but do very different jobs: `variant` (`"primary" | "outline"`) and `grow` (a number).

#### — HOW

```
<Button label="Save Spot" variant="primary" grow={1.3} />
<Button label="Cancel"    variant="outline" grow={1}   />
```

`variant` selects a class from the module CSS. `grow` is applied as an inline `style={{ flex: grow }}`, set per call site.

## — WHY

`variant` answers "what kind of thing is this, visually" — part of the button's own identity. `grow` answers "how much space should this instance take up in this particular row" — a relationship between siblings, decided by whoever is placing the button. Baking `grow` into a CSS class would mean inventing a new variant for every possible width ratio.

### ◆ IDEA TO CONSIDER

The test that separates the two: *"if I pulled this component out of its current parent and dropped it somewhere else, would this prop still make sense?"* `variant` survives that move; `grow` doesn't. Every mature component library draws this exact line.

## 03 Deriving instead of duplicating

### ● PATTERN

### ● BUG

#### KEYWORDS

`Record<K,V>`

single source of truth

## — WHAT

`Badge`'s difficulty color went through a raw `color: string` prop, then a typed `color: ColorVariant` prop passed *alongside* `level`, before landing on computing `color` internally from `level`.

## — HOW

```
const difficultyColor: Record<Difficulty, ColorVariant> = {
  beginner: "green", intermediate: "gold", advanced: "orange", pro: "red",
};
const color = difficultyColor[level]; // computed, never a prop
```

## — WHY

Passing both `level` and `color` as separate props creates two sources of truth that have to agree with each other. Nothing stops a caller from passing `level="beginner"` with `color="red"` — TypeScript checks each prop's type independently, not whether they're consistent.

◆ IDEA TO CONSIDER

`Record<Difficulty, X>` also forces every `Difficulty` value to have an entry — add a new difficulty later without updating the map, and it's a compile error, not a silently-missing color at runtime. Whenever two props always have to move together, that's the signal one of them shouldn't be a prop.

## 04 Reusing domain types instead of bare strings

● CONCEPT

KEYWORDS `union type` `type safety`

— WHAT

`Badge`'s `level` prop uses `Difficulty` (from `src/types/spot.ts`), not `string`.

— HOW

```
level?: Difficulty // not: level?: string
```

— WHY

A bare `string` prop accepts anything — including a typo like `"Beginner"` vs `"beginner"` — that TypeScript can't catch and that would silently fail to match the `difficultyColor` dictionary at runtime. Reusing the domain type gets autocomplete at every call site and a compile error the moment an unsupported value is passed.

◆ IDEA TO CONSIDER

Before typing a new prop as `string`, check whether a matching union type already exists in `src/types/`. If the value is meant to be one of a known, closed set, it should almost never be a bare `string`.

## 05 Optional props as the "one component, multiple roles" switch

### ● PATTERN

#### KEYWORDS

optional prop

presence as a signal

#### — WHAT

`Badge` needed to serve two jobs — a plain type badge and a color-coded difficulty badge — without becoming two components.

#### — HOW

Make `level` optional. Its presence decides which role the badge is playing — passed → colored via the dictionary; omitted → falls through to the shared class's neutral default text color.

#### — WHY

The two badges differ in only one respect (does the text get a level-driven color, or the default color), and the box CSS is identical between them. Forcing two components would duplicate that shared box styling for no reason.

### ◆ IDEA TO CONSIDER

When two "variants" of a UI piece differ in only one small, optional piece of information — not structure — reach for an optional prop before reaching for a second component or a pair of booleans.

## 06 The honest limit — when not to do this

### ● GOTCHA

KEYWORDS

prop soup

structural difference

### — WHAT

This pattern only worked because, in both `Button` and `Badge`, the *only* things that ever differed between instances were small: a class selection, a number, a color.

### — HOW

Ask: "does this only change how it looks, or does it change what it structurally is?" The moment you'd need `if (variant === "x") return <CompletelyDifferentMarkup />` inside one of these, that's the signal to split it back into two components.

### — WHY

If two "variants" actually need different DOM structure, different children, or meaningfully different behavior, cramming them into one component via props usually produces a tangle of conditionals — a "prop soup" component that's harder to read than two small, separate components would have been.

#### ◆ IDEA TO CONSIDER

Every pattern in this manual has a boundary. The skill isn't memorizing "always consolidate" — it's knowing the specific test (look vs. structure) that tells you when consolidation stops paying off.

## A Glossary

**Component sprawl**

A growing set of near-identical components, each duplicating the same structure/CSS for a difference that's actually small.

### Identity prop

A prop describing what a component fundamentally is (e.g. `variant`) — belongs in the component's own CSS.

### Layout prop

A prop describing a component's relationship to its current parent (e.g. `grow`) — usually applied inline by the consumer.

### Single source of truth

The principle that a fact should be stored/computed in exactly one place, with everything else derived from it.

### Prop soup

A component with so many branching props/conditionals that it's harder to read than if it had been split into separate components.

## B Self-test

**Q1** Why does `grow` live as an inline style instead of becoming another `Button` `variant`? ▶

**Q2** What's wrong with a `level` and `color` as separate, individually well-typed, required props? ▶

**Q3** Why does `Record<Difficulty, ColorVariant>` catch a missing-case bug at compile time that a `switch` with `default` wouldn't? ▶

**Q4** What made `level` a good candidate for an optional prop, rather than splitting `Badge` into two components? ▶

**Q5** What's the actual tell that a component should be split into two instead of gaining another variant option? ▶

## C Study links

- ↗ [TypeScript — Record<K,V>](#)
- ↗ [React — Passing props to a component](#)
- ↗ [Radix UI — styling philosophy \(identity vs. consumer-owned layout\)](#)

### SPOT CHECK

SkateSpots Field Manual N°04 · built while consolidating Button and Badge.